Waterpark & Amusement Park Management Software

Complete Technical Blueprint — Linux Server Deployment with PostgreSQL

1. System Architecture Overview

This is a **Monolithic-first, modular architecture** deployed on a Linux server (Ubuntu 22.04 LTS). It is designed to grow into microservices in Phase 3+ without a full rewrite. The system uses a single Next.js full-stack application with PostgreSQL as the primary database, Redis for caching and sessions, and Docker for containerization.

```
┌──────────────────────────────────────────────────────────────┐
│                      INTERNET / USERS                        │
└──────────────────────┬──────────────────────────┬────────────┘
                       │                          │
              [Guest Browser]            [Admin / Staff Browser]
              [Guest Mobile App]
                       │                          │
             ──────────────────────────────────────────────
                          NGINX (Reverse Proxy + SSL)
                       Port 80 → 443, Certbot / Let's Encrypt
             ──────────────────────────────────────────────
                       │                          │
             ┌──────────────────────────────────────────┐
             │        Next.js Application               │
             │  (App Router — Pages + API Routes)       │
             │  Port: 3000 (internal)                   │
             │                                          │
             │  /app        → Guest Portal + Booking    │
             │  /admin      → Admin Dashboard           │
             │  /staff      → Employee Panel            │
             │  /api/v1/    → REST API endpoints        │
             └──────────────────┬───────────────────────┘
                                │
             ┌──────────────────┼──────────────────────────┐
             │                  │                          │
       ┌──────────┐     ┌──────────────┐          ┌──────────────┐
       │PostgreSQL│     │ Redis        │          │ File Storage  │
       │(Primary  │     │ (Sessions,   │          │ (Local /uploads│
       │Database) │     │  Cache,      │          │  or OCI Bucket)│
       │Port 5432 │     │  Queue)      │          └──────────────┘
       └──────────┘     │  Port 6379   │
                        └──────────────┘

External Services (API calls from Next.js):
```

```
┌──────────────────────────────────────────────┐
│  Razorpay (Payments)   |  MSG91 (SMS + WhatsApp) |
│  Gupshup (WhatsApp)    |  SendGrid (Email)       |
│  Firebase (Push Notif) |  Google Maps (Park Map) |
└──────────────────────────────────────────────┘
```

2. Server Requirements

Minimum Production Server (Single VPS / OCI Compute)

Resource	Minimum	Recommended
CPU	2 vCPU	4 vCPU
RAM	4 GB	8 GB
Storage	50 GB SSD	100 GB SSD
OS	Ubuntu 22.04 LTS	Ubuntu 22.04 LTS
Network	1 Gbps	1 Gbps
Bandwidth	2 TB/month	5 TB/month

Software Stack on Server

Software	Version	Purpose
Ubuntu	22.04 LTS	Operating System
Docker	26.x	Containerization
Docker Compose	2.x	Multi-container management
Nginx	1.24+	Reverse proxy + SSL termination
Certbot	Latest	Let's Encrypt SSL certificates
Node.js	20 LTS (in Docker)	Next.js runtime
PostgreSQL	16 (in Docker)	Primary database
Redis	7 (in Docker)	Cache + session store
GitHub Actions	Cloud	CI/CD pipeline

3. Project Folder Structure

```
/waterpark-app/
├── docker-compose.yml            # Main compose file (production)
├── docker-compose.dev.yml        # Dev overrides
├── .env                         # Environment variables (NEVER commit)
├── .env.example                 # Template for .env
├── nginx/
```

```
│   ├── nginx.conf                # Main Nginx config
│   └── conf.d/
│       └── waterpark.conf        # App virtual host config
│
├── postgres/
│   ├── init/
│   │   └── 01-init.sql           # DB init script (runs on first start)
│   └── backups/                  # Auto backup destination
│
├── app/                          # Next.js Application
│   ├── package.json
│   ├── next.config.js
│   ├── tsconfig.json
│   ├── Dockerfile                # Production Docker image
│   ├── Dockerfile.dev            # Dev Docker image
│   │
│   ├── prisma/                   # Prisma ORM
│   │   ├── schema.prisma         # Full database schema
│   │   └── migrations/           # Auto-generated migrations
│   │
│   ├── src/
│   │   ├── app/                  # Next.js App Router
│   │   │   ├── layout.tsx        # Root layout
│   │   │   ├── page.tsx          # Landing page
│   │   │   │
│   │   │   ├── (guest)/          # Guest-facing routes
│   │   │   │   ├── booking/      # Ticket booking flow
│   │   │   │   ├── my-account/   # Guest profile, bookings, loyalty
│   │   │   │   ├── park-info/    # Park map, rides, timing
│   │   │   │   └── payment/      # Payment + Manual UPI flow
│   │   │   │
│   │   │   ├── (admin)/          # Admin routes (protected)
│   │   │   │   ├── dashboard/    # Admin home dashboard
│   │   │   │   ├── bookings/     # All bookings management
│   │   │   │   ├── tickets/      # Ticket types & pricing
│   │   │   │   ├── rides/        # Ride management
│   │   │   │   ├── food/         # F&B management
│   │   │   │   ├── lockers/      # Locker management
│   │   │   │   ├── staff/        # Staff management
│   │   │   │   ├── crm/          # Guest CRM + Leads
│   │   │   │   ├── campaigns/    # Marketing campaigns
│   │   │   │   ├── payments/     # Payment master + UPI queue
│   │   │   │   ├── maintenance/  # CMMS - ride maintenance
│   │   │   │   ├── reports/      # Analytics + reports
│   │   │   │   └── settings/     # System configuration
│   │   │   │
│   │   │   ├── (staff)/          # Employee routes (protected)
│   │   │   │   ├── pos/          # POS terminal
│   │   │   │   ├── rides/        # Ride operator panel
│   │   │   │   ├── food/         # F&B staff panel
│   │   │   │   ├── lockers/      # Locker attendant panel
│   │   │   │   ├── maintenance/  # Technician panel
│   │   │   │   └── security/     # Security panel
│   │   │   │
│   │   │   └── api/              # API routes
│   │   │       └── v1/
```

```
│   │   │           ├── auth/       # Login, OTP, refresh tokens
│   │   │           ├── bookings/   # Booking CRUD
│   │   │           ├── tickets/    # Ticket types
│   │   │           ├── rides/      # Ride data
│   │   │           ├── food/       # F&B + inventory
│   │   │           ├── lockers/    # Locker management
│   │   │           ├── payments/   # Payment processing
│   │   │           │   ├── razorpay/  webhook handler
│   │   │           │   └── manual-upi/ UTR verification
│   │   │           ├── crm/        # Guests + leads
│   │   │           ├── campaigns/  # WhatsApp/SMS campaigns
│   │   │           ├── staff/      # Staff management
│   │   │           ├── maintenance/ # Work orders
│   │   │           ├── reports/    # Analytics queries
│   │   │           ├── webhooks/   # External webhooks
│   │   │           │   ├── razorpay/
│   │   │           │   ├── whatsapp/
│   │   │           │   └── sms/
│   │   │           └── upload/     # File upload handler
│   │   │
│   │   ├── components/            # Reusable UI components
│   │   │   ├── ui/                # Base UI (buttons, inputs, modals)
│   │   │   ├── admin/             # Admin-specific components
│   │   │   ├── staff/             # Staff-specific components
│   │   │   ├── guest/             # Guest-facing components
│   │   │   └── shared/            # Shared across all roles
│   │   │
│   │   ├── lib/                   # Core libraries
│   │   │   ├── db.ts              # Prisma client singleton
│   │   │   ├── auth.ts            # NextAuth config + RBAC
│   │   │   ├── redis.ts           # Redis client
│   │   │   ├── razorpay.ts        # Razorpay SDK wrapper
│   │   │   ├── msg91.ts           # MSG91 SMS + WhatsApp
│   │   │   ├── gupshup.ts         # Gupshup WhatsApp API
│   │   │   ├── sendgrid.ts        # Email service
│   │   │   ├── firebase.ts        # Push notifications
│   │   │   └── utils.ts           # Helpers, formatters
│   │   │
│   │   ├── middleware.ts           # Auth middleware (route protection)
│   │   │
│   │   └── types/                 # TypeScript types
│   │       ├── auth.ts
│   │       ├── booking.ts
│   │       ├── payment.ts
│   │       └── crm.ts
│   │
│   └── public/
│       ├── park-map.svg
│       └── assets/
│
└── scripts/
    ├── backup-db.sh               # PostgreSQL backup script
    ├── restore-db.sh              # PostgreSQL restore
    └── deploy.sh                  # Deployment script
```

4. Complete PostgreSQL Database Schema (Prisma)

```
// prisma/schema.prisma
// Waterpark & Amusement Park Management System
// PostgreSQL + Prisma ORM

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

// ═══════════════════════════════════════════════════════════
// ENUMS
// ═══════════════════════════════════════════════════════════

enum UserRole {
  ADMIN
  EMPLOYEE
  USER
}

enum EmployeeSubRole {
  TICKET_COUNTER
  FB_STAFF
  RIDE_OPERATOR
  MAINTENANCE_TECH
  LOCKER_ATTENDANT
  SALES_EXECUTIVE
  SECURITY_STAFF
  EVENT_COORDINATOR
}

enum BookingStatus {
  PENDING
  CONFIRMED
  CHECKED_IN
  COMPLETED
  CANCELLED
  REFUNDED
}

enum PaymentMethod {
  GATEWAY
  MANUAL_UPI
  CASH
  WRISTBAND
  COMPLIMENTARY
}

enum PaymentStatus {
  PENDING
  PAID
```

```
  FAILED
  REFUNDED
  REJECTED
}

enum GatewayMode {
  TEST
  LIVE
}

enum RideStatus {
  ACTIVE
  MAINTENANCE
  CLOSED
  SEASONAL
}

enum WorkOrderPriority {
  CRITICAL
  HIGH
  MEDIUM
  LOW
}

enum WorkOrderStatus {
  OPEN
  IN_PROGRESS
  COMPLETED
  CANCELLED
}

enum LeadStage {
  NEW
  CONTACTED
  INTERESTED
  PROPOSAL_SENT
  BOOKED
  LOST
}

enum CampaignChannel {
  WHATSAPP
  SMS
  EMAIL
  PUSH
}

enum CampaignType {
  PROMOTIONAL
  TRANSACTIONAL
  BIRTHDAY
  REENGAGEMENT
  LEAD_NURTURE
}

enum CampaignStatus {
```

```
  DRAFT
  SCHEDULED
  RUNNING
  COMPLETED
  PAUSED
}

enum LockerStatus {
  AVAILABLE
  OCCUPIED
  MAINTENANCE
}

enum NotificationChannel {
  WHATSAPP
  SMS
  EMAIL
  PUSH
}

// ═══════════════════════════════════════════════════════════
// USERS
// ═══════════════════════════════════════════════════════════

model User {
  id               String    @id @default(cuid())
  name             String
  email            String?   @unique
  mobile           String    @unique
  mobileVerified    Boolean   @default(false)
  passwordHash     String?
  role             UserRole  @default(USER)
  isActive         Boolean   @default(true)
  createdAt        DateTime  @default(now())
  updatedAt        DateTime  @updatedAt

  // Relations
  employeeProfile   EmployeeProfile?
  guestProfile      GuestProfile?
  bookings          Booking[]
  transactions      Transaction[]
  assignedLeads     Lead[]           @relation("AssignedLeads")
  verifiedPayments  Transaction[]     @relation("VerifiedPayments")
  workOrdersAssigned WorkOrder[]      @relation("AssignedTechnician")
  sessions          Session[]
  activityLogs      ActivityLog[]

  @@map("users")
}

model Session {
  id        String   @id @default(cuid())
  userId    String
  token     String   @unique
  expiresAt DateTime
  ipAddress String?
```

```
  userAgent String?
  createdAt DateTime @default(now())

  user      User     @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@index([userId])
  @@index([token])
  @@map("sessions")
}

// ═══════════════════════════════════════════════════════════
// EMPLOYEE PROFILE
// ═══════════════════════════════════════════════════════════

model EmployeeProfile {
  id             String          @id @default(cuid())
  userId         String          @unique
  subRole        EmployeeSubRole
  department     String
  employeeCode   String          @unique
  joiningDate    DateTime
  permissions    Json            @default("{}")
  certifications Certification[]
  canVerifyUpi   Boolean         @default(false)
  shifts         Shift[]
  attendance     Attendance[]
  createdAt      DateTime        @default(now())
  updatedAt      DateTime        @updatedAt

  user           User            @relation(fields: [userId], references: [id], onDelete

  @@map("employee_profiles")
}

model Certification {
  id              String         @id @default(cuid())
  employeeId      String
  name            String         // "Lifeguard", "First Aid", "Ride Operator"
  issuedDate      DateTime
  expiryDate      DateTime
  issuingAuthority String
  documentUrl     String?
  isExpired       Boolean        @default(false)

  employee        EmployeeProfile @relation(fields: [employeeId], references: [id], o

  @@index([employeeId])
  @@index([expiryDate])
  @@map("certifications")
}

// ═══════════════════════════════════════════════════════════
// GUEST PROFILE (CRM Core)
// ═══════════════════════════════════════════════════════════

model GuestProfile {
```

```
  id                  String    @id @default(cuid())
  userId              String    @unique
  dob                 DateTime?
  gender              String?
  city                String?
  state               String?
  pincode             String?
  addressLine         String?

  // Acquisition
  acquisitionSource   String?   // google, facebook, referral, walk-in
  utmCampaign         String?
  utmMedium           String?
  utmSource           String?
  utmContent          String?
  referredByUserId    String?
  referralCode        String    @unique @default(cuid())

  // Opt-ins
  whatsappOptIn       Boolean   @default(false)
  smsOptIn            Boolean   @default(false)
  emailOptIn          Boolean   @default(false)
  pushOptIn           Boolean   @default(false)
  optInUpdatedAt      DateTime?

  // Loyalty
  loyaltyTier         String    @default("BRONZE")  // BRONZE, SILVER, GOLD, PLATINUM
  loyaltyPoints       Int       @default(0)
  lifetimePoints      Int       @default(0)

  // Stats (denormalized for speed)
  totalVisits         Int       @default(0)
  lifetimeSpend       Decimal   @default(0) @db.Decimal(10,2)
  avgSpendPerVisit    Decimal   @default(0) @db.Decimal(10,2)
  lastVisitDate       DateTime?
  firstVisitDate      DateTime?

  // NPS
  npsScore            Int?
  npsRespondedAt      DateTime?

  // Tags
  tags                String[]  @default([])

  createdAt           DateTime  @default(now())
  updatedAt           DateTime  @updatedAt

  // Relations
  user                User            @relation(fields: [userId], references: [id], onDe
  children            GuestChild[]
  loyaltyTransactions LoyaltyTransaction[]
  feedbacks           Feedback[]
  communicationLogs   CommunicationLog[]
  wristbands          Wristband[]
  referrals           GuestProfile[]   @relation("Referrals")
  referredBy          GuestProfile?    @relation("Referrals", fields: [referredByUserId
```

```
  @@index([mobile: false])   // indirect via user
  @@index([loyaltyTier])
  @@index([lastVisitDate])
  @@index([referralCode])
  @@map("guest_profiles")
}

model GuestChild {
  id             String      @id @default(cuid())
  guestProfileId  String
  name           String
  dob            DateTime
  gender         String?
  wristbandId    String?
  photoUrl       String?
  createdAt      DateTime    @default(now())

  guestProfile   GuestProfile @relation(fields: [guestProfileId], references: [id], on

  @@index([guestProfileId])
  @@map("guest_children")
}

// ═══════════════════════════════════════════════════════════
// PARK CONFIGURATION
// ═══════════════════════════════════════════════════════════

model ParkConfig {
  id             String   @id @default(cuid())
  key            String   @unique
  value          String
  description    String?
  updatedAt      DateTime @updatedAt

  @@map("park_config")
}

model Zone {
  id             String   @id @default(cuid())
  name           String
  description    String?
  capacity       Int
  isActive       Boolean  @default(true)
  rides          Ride[]
  lockers        Locker[]

  @@map("zones")
}

// ═══════════════════════════════════════════════════════════
// TICKETS & PRICING
// ═══════════════════════════════════════════════════════════

model TicketType {
  id             String    @id @default(cuid())
```

```
  name            String
  description     String?
  basePrice       Decimal   @db.Decimal(10,2)
  category        String    // GENERAL, VIP, GROUP, FAMILY, CORPORATE, SEASON_PASS
  validDays       Int?      // null = single day
  maxGuests       Int?
  includesRides   Boolean   @default(true)
  rideCredits     Int?      // null = unlimited
  includesFB      Boolean   @default(false)
  fbCredits       Decimal?  @db.Decimal(10,2)
  includesLocker  Boolean   @default(false)
  isActive        Boolean   @default(true)
  sortOrder       Int       @default(0)
  createdAt       DateTime  @default(now())
  updatedAt       DateTime  @updatedAt

  pricingRules    PricingRule[]
  bookingItems    BookingItem[]

  @@map("ticket_types")
}

model PricingRule {
  id              String     @id @default(cuid())
  ticketTypeId    String
  name            String
  type            String     // SURGE, EARLY_BIRD, GROUP_DISCOUNT, PROMO
  discountType    String     // PERCENTAGE, FLAT
  discountValue   Decimal    @db.Decimal(10,2)
  minGuests       Int?
  maxGuests       Int?
  validFrom       DateTime?
  validTo         DateTime?
  daysOfWeek      Int[]      // 0=Sun, 1=Mon...6=Sat
  isActive        Boolean    @default(true)

  ticketType      TicketType @relation(fields: [ticketTypeId], references: [id])

  @@map("pricing_rules")
}

// ═══════════════════════════════════════════════════════════
// BOOKINGS
// ═══════════════════════════════════════════════════════════

model Booking {
  id              String        @id @default(cuid())
  bookingRef      String        @unique @default(cuid())
  userId          String
  guestName       String
  guestMobile     String
  guestEmail      String?
  visitDate       DateTime
  visitSlot       String?       // "09:00", "12:00", "15:00"
  totalGuests     Int
  adultsCount     Int           @default(0)
```

```
  childrenCount   Int          @default(0)
  status          BookingStatus @default(PENDING)
  subtotal        Decimal       @db.Decimal(10,2)
  discount        Decimal       @default(0) @db.Decimal(10,2)
  taxAmount       Decimal       @default(0) @db.Decimal(10,2)
  totalAmount     Decimal       @db.Decimal(10,2)
  notes           String?
  qrCode          String?       // generated QR code value
  checkedInAt     DateTime?
  checkedInByStaff String?
  source          String?       // ONLINE, COUNTER, KIOSK
  createdAt       DateTime      @default(now())
  updatedAt       DateTime      @updatedAt

  user            User         @relation(fields: [userId], references: [id])
  bookingItems    BookingItem[]
  transactions    Transaction[]
  addons          BookingAddon[]

  @@index([userId])
  @@index([visitDate])
  @@index([status])
  @@index([bookingRef])
  @@map("bookings")
}

model BookingItem {
  id              String     @id @default(cuid())
  bookingId       String
  ticketTypeId    String
  quantity        Int
  unitPrice       Decimal    @db.Decimal(10,2)
  discount        Decimal    @default(0) @db.Decimal(10,2)
  totalPrice      Decimal    @db.Decimal(10,2)

  booking         Booking    @relation(fields: [bookingId], references: [id], onDelete:
  ticketType      TicketType @relation(fields: [ticketTypeId], references: [id])

  @@map("booking_items")
}

model BookingAddon {
  id              String   @id @default(cuid())
  bookingId       String
  addonType       String   // LOCKER, CABANA, MEAL, FASTPASS, PHOTO_PKG
  addonName       String
  quantity        Int
  unitPrice       Decimal  @db.Decimal(10,2)
  totalPrice      Decimal  @db.Decimal(10,2)
  details         Json?

  booking         Booking  @relation(fields: [bookingId], references: [id], onDelete: C

  @@map("booking_addons")
}
```

```
// ═══════════════════════════════════════════════════════════
// PAYMENTS & TRANSACTIONS
// ═══════════════════════════════════════════════════════════

model PaymentGateway {
  id                    String      @id @default(cuid())
  name                  String      // RAZORPAY, CASHFREE, PAYU
  displayName           String
  keyId                 String
  keySecretEncrypted    String
  webhookSecretEncrypted String?
  mode                  GatewayMode @default(LIVE)
  isActive              Boolean     @default(false)
  isPrimary             Boolean     @default(false)
  allowedMethods        Json        @default("[]") // ["upi","card","netbanking","wallet"
  minAmount             Decimal?    @db.Decimal(10,2)
  maxAmount             Decimal?    @db.Decimal(10,2)
  createdAt             DateTime    @default(now())
  updatedAt             DateTime    @updatedAt

  @@map("payment_gateways")
}

model ManualUpiAccount {
  id                  String   @id @default(cuid())
  displayName         String
  upiId               String
  accountName         String
  qrImageUrl          String?
  minAmount           Decimal? @db.Decimal(10,2)
  maxAmount           Decimal? @db.Decimal(10,2)
  verificationRole    String   @default("ADMIN") // ADMIN or EMPLOYEE
  autoRejectMinutes   Int      @default(120)
  autoRemindMinutes   Int      @default(15)
  showOnline          Boolean  @default(true)
  showCounter         Boolean  @default(true)
  showKiosk           Boolean  @default(true)
  isActive            Boolean  @default(true)
  createdAt           DateTime @default(now())
  updatedAt           DateTime @updatedAt

  transactions        Transaction[]

  @@map("manual_upi_accounts")
}

model Transaction {
  id                  String        @id @default(cuid())
  bookingId           String?
  userId              String
  amount              Decimal       @db.Decimal(10,2)
  currency            String        @default("INR")
  paymentMethod       PaymentMethod
  gatewayName         String?
  gatewayOrderId      String?
  gatewayPaymentId    String?
```

```
  manualUpiAccountId  String?
  utrNumber           String?       @unique
  status              PaymentStatus  @default(PENDING)
  verifiedByUserId    String?
  verifiedAt          DateTime?
  ipAddress           String?
  deviceInfo          String?
  notes               String?
  refundAmount        Decimal?      @db.Decimal(10,2)
  refundedAt          DateTime?
  createdAt           DateTime      @default(now())
  updatedAt           DateTime      @updatedAt

  booking             Booking?       @relation(fields: [bookingId], references: [id])
  user                User          @relation(fields: [userId], references: [id])
  verifiedBy          User?         @relation("VerifiedPayments", fields: [verifiedByU
  manualUpiAccount    ManualUpiAccount? @relation(fields: [manualUpiAccountId], referenc
  upiAuditLog         ManualUpiAuditLog?

  @@index([bookingId])
  @@index([userId])
  @@index([status])
  @@index([utrNumber])
  @@index([createdAt])
  @@map("transactions")
}

model ManualUpiAuditLog {
  id             String   @id @default(cuid())
  transactionId   String   @unique
  submittedUtr    String
  submittedAt     DateTime
  verifiedByUserId String?
  action         String?  // APPROVED or REJECTED
  actionAt       DateTime?
  flagReason      String?
  notes          String?

  transaction     Transaction @relation(fields: [transactionId], references: [id])

  @@map("manual_upi_audit_logs")
}

// ═══════════════════════════════════════════════════════════
// RIDES & ATTRACTIONS
// ═══════════════════════════════════════════════════════════

model Ride {
  id               String     @id @default(cuid())
  name             String
  description       String?
  category         String     // WATER_SLIDE, ROLLER_COASTER, FAMILY, KIDS, THRILL
  zoneId           String?
  status           RideStatus @default(ACTIVE)
  capacityPerCycle  Int
  cyclesPerHour     Int?
```

```
  minAgeYears      Int?
  maxAgeYears      Int?
  minHeightCm      Int?
  maxHeightCm      Int?
  minWeightKg      Int?
  maxWeightKg      Int?
  isPaidRide       Boolean   @default(false)
  rideCredits      Int?
  locationX        Float?    // Park map coordinates
  locationY        Float?
  photoUrl         String?
  safetyNotes      String?
  createdAt        DateTime  @default(now())
  updatedAt        DateTime  @updatedAt

  zone             Zone?     @relation(fields: [zoneId], references: [id])
  maintenanceAsset MaintenanceAsset?
  incidents        Incident[]
  rideAccessLogs   RideAccessLog[]

  @@map("rides")
}

model RideAccessLog {
  id             String   @id @default(cuid())
  rideId         String
  wristbandId    String?
  userId         String?
  accessAt       DateTime @default(now())
  isValid        Boolean  @default(true)
  rejectionReason String?

  ride           Ride    @relation(fields: [rideId], references: [id])

  @@index([rideId])
  @@index([accessAt])
  @@map("ride_access_logs")
}

// ═══════════════════════════════════════════════════════════
// WRISTBANDS (RFID)
// ═══════════════════════════════════════════════════════════

model Wristband {
  id             String      @id @default(cuid())
  rfidCode       String      @unique
  guestProfileId String?
  bookingId      String?
  color          String?     // Color-coded by ticket type
  walletBalance  Decimal     @default(0) @db.Decimal(10,2)
  issuedAt       DateTime    @default(now())
  returnedAt     DateTime?
  isActive       Boolean     @default(true)
  depositAmount  Decimal?    @db.Decimal(10,2)
  depositRefunded Boolean    @default(false)
```

```
  guestProfile    GuestProfile? @relation(fields: [guestProfileId], references: [id])
  walletTopups    WristbandTopup[]

  @@index([rfidCode])
  @@map("wristbands")
}

model WristbandTopup {
  id           String    @id @default(cuid())
  wristbandId   String
  amount       Decimal   @db.Decimal(10,2)
  method        String    // CASH, GATEWAY, ONLINE
  transactionId String?
  topupAt      DateTime  @default(now())
  staffId      String?

  wristband     Wristband @relation(fields: [wristbandId], references: [id])

  @@map("wristband_topups")
}

// ═══════════════════════════════════════════════════════════
// F&B (FOOD & BEVERAGE)
// ═══════════════════════════════════════════════════════════

model FBOutlet {
  id          String       @id @default(cuid())
  name        String
  location    String?
  type        String       // RESTAURANT, SNACK_COUNTER, KIOSK, POOLSIDE
  isActive    Boolean      @default(true)
  menuItems   FBMenuItem[]
  orders      FBOrder[]

  @@map("fb_outlets")
}

model FBMenuItem {
  id           String   @id @default(cuid())
  outletId     String
  name         String
  description   String?
  category     String
  price        Decimal  @db.Decimal(10,2)
  isVeg        Boolean  @default(true)
  allergens    String[]
  isAvailable   Boolean  @default(true)
  stockQty     Int?
  minStockAlert Int?

  outlet       FBOutlet    @relation(fields: [outletId], references: [id])
  orderItems    FBOrderItem[]

  @@index([outletId])
  @@map("fb_menu_items")
}
```

```
model FBOrder {
  id           String      @id @default(cuid())
  outletId     String
  wristbandId  String?
  userId       String?
  bookingId    String?
  orderType    String      // DINE_IN, TAKEAWAY, DELIVERY
  status       String      @default("PENDING") // PENDING, PREPARING, READY, DELIVERED,
  subtotal     Decimal     @db.Decimal(10,2)
  discount     Decimal     @default(0) @db.Decimal(10,2)
  totalAmount  Decimal     @db.Decimal(10,2)
  paymentMethod String?
  transactionId String?
  tableNumber  String?
  notes        String?
  createdAt    DateTime    @default(now())
  updatedAt    DateTime    @updatedAt

  outlet       FBOutlet    @relation(fields: [outletId], references: [id])
  items        FBOrderItem[]

  @@index([outletId])
  @@index([createdAt])
  @@map("fb_orders")
}

model FBOrderItem {
  id         String    @id @default(cuid())
  orderId    String
  menuItemId String
  quantity   Int
  unitPrice  Decimal   @db.Decimal(10,2)
  totalPrice Decimal   @db.Decimal(10,2)
  notes      String?

  order      FBOrder   @relation(fields: [orderId], references: [id])
  menuItem   FBMenuItem @relation(fields: [menuItemId], references: [id])

  @@map("fb_order_items")
}

// ═══════════════════════════════════════════════════════════
// LOCKERS
// ═══════════════════════════════════════════════════════════

model Locker {
  id           String      @id @default(cuid())
  lockerNumber String      @unique
  zoneId       String?
  size         String      // SMALL, MEDIUM, LARGE, XL
  status       LockerStatus @default(AVAILABLE)
  hourlyRate   Decimal     @db.Decimal(10,2)
  fullDayRate  Decimal     @db.Decimal(10,2)
  currentRental LockerRental? @relation("CurrentRental")
  rentals      LockerRental[]
```

```
  zone         Zone?       @relation(fields: [zoneId], references: [id])

  @@map("lockers")
}

model LockerRental {
  id           String   @id @default(cuid())
  lockerId      String
  userId        String?
  wristbandId   String?
  rentalType    String   // HOURLY, FULL_DAY
  startAt       DateTime @default(now())
  endAt         DateTime?
  rateApplied   Decimal  @db.Decimal(10,2)
  deposit       Decimal  @db.Decimal(10,2)
  depositRefunded Boolean @default(false)
  status        String   @default("ACTIVE") // ACTIVE, RETURNED, OVERDUE
  transactionId String?

  locker        Locker   @relation(fields: [lockerId], references: [id])
  activeLocker  Locker?  @relation("CurrentRental", fields: [lockerId], references: [id

  @@index([lockerId])
  @@map("locker_rentals")
}

// ═══════════════════════════════════════════════════════════
// MAINTENANCE (CMMS)
// ═══════════════════════════════════════════════════════════

model MaintenanceAsset {
  id             String     @id @default(cuid())
  rideId         String     @unique
  model          String?
  manufacturer    String?
  serialNumber    String?
  installDate     DateTime?
  warrantyExpiry  DateTime?
  totalCycles     Int       @default(0)
  lastServiceDate DateTime?
  nextServiceDate DateTime?
  documents       Json      @default("[]")

  ride           Ride      @relation(fields: [rideId], references: [id])
  workOrders      WorkOrder[]
  inspections     SafetyInspection[]

  @@map("maintenance_assets")
}

model WorkOrder {
  id             String           @id @default(cuid())
  assetId        String
  title          String
  description     String
```

```
  priority        WorkOrderPriority @default(MEDIUM)
  status          WorkOrderStatus   @default(OPEN)
  assignedToId    String?
  reportedById    String?
  scheduledAt     DateTime?
  startedAt       DateTime?
  completedAt     DateTime?
  partsUsed       Json              @default("[]")
  laborHours      Float?
  photos          String[]          @default([])
  notes           String?
  createdAt       DateTime          @default(now())
  updatedAt       DateTime          @updatedAt

  asset           MaintenanceAsset  @relation(fields: [assetId], references: [id])
  assignedTo      User?             @relation("AssignedTechnician", fields: [assignedToI

  @@index([assetId])
  @@index([status])
  @@map("work_orders")
}

model SafetyInspection {
  id              String           @id @default(cuid())
  assetId         String
  inspectorId     String
  inspectionDate  DateTime         @default(now())
  type            String           // DAILY, WEEKLY, MONTHLY, ANNUAL
  checklistData   Json             // [{item, status, notes}]
  overallStatus   String           // PASS, FAIL, CONDITIONAL
  eSignature      String?
  notes           String?
  nextInspectionDate DateTime?

  asset           MaintenanceAsset @relation(fields: [assetId], references: [id])

  @@index([assetId])
  @@index([inspectionDate])
  @@map("safety_inspections")
}

model Incident {
  id              String   @id @default(cuid())
  rideId          String?
  reportedById    String
  title           String
  description     String
  severity        String   // CRITICAL, HIGH, MEDIUM, LOW
  category        String   // INJURY, MALFUNCTION, THEFT, FIRE, MISSING_CHILD
  status          String   @default("OPEN")
  witnessCount    Int?
  photos          String[] @default([])
  correctiveAction String?
  closedAt        DateTime?
  createdAt       DateTime @default(now())
```

```
  ride           Ride?    @relation(fields: [rideId], references: [id])

  @@index([rideId])
  @@map("incidents")
}

// ═══════════════════════════════════════════════════════════
// CRM - LEADS
// ═══════════════════════════════════════════════════════════

model Lead {
  id                 String    @id @default(cuid())
  name               String
  mobile             String
  email              String?
  source             String    // WEBSITE, WHATSAPP, REFERRAL, WALKIN, CORPORATE
  eventType          String?   // BIRTHDAY, CORPORATE, SCHOOL, FAMILY
  groupSize          Int?
  tentativeDate      DateTime?
  budgetRange        String?
  stage              LeadStage @default(NEW)
  leadScore          Int      @default(0)
  assignedToId       String?
  nextFollowupAt     DateTime?
  lostReason         String?
  convertedBookingId  String?
  notes              String?
  tags               String[]  @default([])
  createdAt          DateTime  @default(now())
  updatedAt          DateTime  @updatedAt

  assignedTo         User?    @relation("AssignedLeads", fields: [assignedToId], refer
  activities         LeadActivity[]

  @@index([stage])
  @@index([assignedToId])
  @@map("leads")
}

model LeadActivity {
  id         String   @id @default(cuid())
  leadId     String
  type       String   // CALL, WHATSAPP, EMAIL, MEETING, NOTE
  description String
  doneById    String
  doneAt     DateTime @default(now())
  nextAction  String?
  nextActionAt DateTime?

  lead       Lead    @relation(fields: [leadId], references: [id], onDelete: Cascade)

  @@index([leadId])
  @@map("lead_activities")
}

// ═══════════════════════════════════════════════════════════
```

```
// COMMUNICATION & CAMPAIGNS
// ═══════════════════════════════════════════════════════════

model CommunicationGateway {
  id                String   @id @default(cuid())
  channel           NotificationChannel
  provider          String
  apiKeyEncrypted   String
  senderId          String?
  webhookUrl        String?
  isActive          Boolean  @default(false)
  isPrimary         Boolean  @default(false)
  configJson        Json     @default("{}")
  createdAt         DateTime @default(now())
  updatedAt         DateTime @updatedAt

  @@unique([channel, isPrimary])
  @@map("communication_gateways")
}

model MessageTemplate {
  id                  String          @id @default(cuid())
  channel             NotificationChannel
  category            String          // UTILITY, MARKETING, AUTHENTICATION
  name                String
  content             String
  variables           Json            @default("[]")
  dltTemplateId       String?
  waTemplateId        String?
  waTemplateName      String?
  isApproved          Boolean         @default(false)
  isActive            Boolean         @default(true)
  createdAt           DateTime        @default(now())

  campaignUsages      Campaign[]

  @@map("message_templates")
}

model Campaign {
  id                  String         @id @default(cuid())
  name                String
  channel             CampaignChannel
  type                CampaignType
  segmentId           String?
  templateId          String?
  customMessage       String?
  scheduleAt          DateTime?
  status              CampaignStatus @default(DRAFT)
  sentCount           Int            @default(0)
  deliveredCount      Int            @default(0)
  readCount           Int            @default(0)
  clickCount          Int            @default(0)
  optOutCount         Int            @default(0)
  revenueAttributed   Decimal        @default(0) @db.Decimal(10,2)
  createdById         String
```

```
  createdAt          DateTime      @default(now())
  completedAt        DateTime?

  template           MessageTemplate? @relation(fields: [templateId], references: [id])
  segment            GuestSegment?    @relation(fields: [segmentId], references: [id])

  @@map("campaigns")
}

model GuestSegment {
  id             String     @id @default(cuid())
  name           String
  description     String?
  filterRules     Json       // AND/OR conditions
  guestCount      Int        @default(0)
  lastRefreshed   DateTime?
  isStatic        Boolean    @default(false)
  createdById     String
  createdAt       DateTime   @default(now())

  campaigns       Campaign[]

  @@map("guest_segments")
}

model CommunicationLog {
  id               String             @id @default(cuid())
  guestProfileId    String?
  channel          NotificationChannel
  templateId        String?
  campaignId        String?
  messageContent    String
  status           String             // SENT, DELIVERED, READ, FAILED, OPTED_OUT
  providerMessageId String?
  sentAt           DateTime?
  deliveredAt       DateTime?
  readAt           DateTime?
  failedAt         DateTime?
  errorReason       String?

  guestProfile      GuestProfile? @relation(fields: [guestProfileId], references: [id])

  @@index([guestProfileId])
  @@index([sentAt])
  @@map("communication_logs")
}

// ═══════════════════════════════════════════════════════════
// AUTOMATION WORKFLOWS
// ═══════════════════════════════════════════════════════════

model AutomationWorkflow {
  id             String   @id @default(cuid())
  name           String
  triggerEvent    String   // BOOKING_CONFIRMED, VISIT_COMPLETED, BIRTHDAY, etc.
  conditions      Json     @default("{}")
```

```
  delayMinutes    Int      @default(0)
  channel         NotificationChannel
  templateId      String?
  isActive        Boolean  @default(true)
  executionCount  Int      @default(0)
  createdAt       DateTime @default(now())

  @@map("automation_workflows")
}

// ═══════════════════════════════════════════════════════════
// LOYALTY
// ═══════════════════════════════════════════════════════════

model LoyaltyTransaction {
  id              String       @id @default(cuid())
  guestProfileId  String
  type            String       // EARN, REDEEM, EXPIRE, BONUS
  points          Int
  description     String
  referenceId     String?      // booking/transaction ID
  balance         Int
  createdAt       DateTime     @default(now())

  guestProfile    GuestProfile @relation(fields: [guestProfileId], references: [id])

  @@index([guestProfileId])
  @@map("loyalty_transactions")
}

// ═══════════════════════════════════════════════════════════
// FEEDBACK
// ═══════════════════════════════════════════════════════════

model Feedback {
  id              String       @id @default(cuid())
  guestProfileId  String?
  bookingId       String?
  overallRating   Int          // 1-5
  rideRating      Int?
  fbRating        Int?
  staffRating     Int?
  cleanlinessRating Int?
  npsScore        Int?         // 0-10
  comments        String?
  source          String       @default("POST_VISIT") // POST_VISIT, IN_PARK, GOOGLE
  createdAt       DateTime     @default(now())

  guestProfile    GuestProfile? @relation(fields: [guestProfileId], references: [id])

  @@map("feedbacks")
}

// ═══════════════════════════════════════════════════════════
// STAFF MANAGEMENT
// ═══════════════════════════════════════════════════════════
```

```
model Shift {
  id             String         @id @default(cuid())
  employeeId      String
  date           DateTime
  startTime       String         // "09:00"
  endTime         String         // "18:00"
  zone           String?
  role           String?
  notes          String?
  createdAt       DateTime       @default(now())

  employee        EmployeeProfile @relation(fields: [employeeId], references: [id])
  attendance      Attendance[]

  @@index([employeeId])
  @@index([date])
  @@map("shifts")
}

model Attendance {
  id             String         @id @default(cuid())
  employeeId      String
  shiftId        String?
  date           DateTime
  checkIn        DateTime?
  checkOut       DateTime?
  status         String         @default("PRESENT") // PRESENT, ABSENT, LATE, HALF_DAY
  notes          String?

  employee        EmployeeProfile @relation(fields: [employeeId], references: [id])
  shift          Shift?         @relation(fields: [shiftId], references: [id])

  @@unique([employeeId, date])
  @@map("attendance")
}

// ═══════════════════════════════════════════════════════════
// AUDIT LOG
// ═══════════════════════════════════════════════════════════

model ActivityLog {
  id         String   @id @default(cuid())
  userId     String
  action     String
  module      String
  recordId    String?
  oldValues   Json?
  newValues   Json?
  ipAddress   String?
  createdAt   DateTime @default(now())

  user       User     @relation(fields: [userId], references: [id])

  @@index([userId])
  @@index([module])
```

```
  @@index([createdAt])
  @@map("activity_logs")
}
```

5. Docker Compose Configuration

```
# docker-compose.yml
version: "3.9"

services:

  postgres:
    image: postgres:16-alpine
    container_name: waterpark_postgres
    restart: unless-stopped
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./postgres/init:/docker-entrypoint-initdb.d
    ports:
      - "127.0.0.1:5432:5432"   # bind only to localhost
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
      interval: 10s
      timeout: 5s
      retries: 5

  redis:
    image: redis:7-alpine
    container_name: waterpark_redis
    restart: unless-stopped
    command: redis-server --requirepass ${REDIS_PASSWORD} --maxmemory 512mb --maxmemor
    volumes:
      - redis_data:/data
    ports:
      - "127.0.0.1:6379:6379"   # bind only to localhost
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
      timeout: 5s
      retries: 5

  app:
    build:
      context: ./app
      dockerfile: Dockerfile
    container_name: waterpark_app
    restart: unless-stopped
    environment:
      - DATABASE_URL=postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@postgres:5432/${
      - REDIS_URL=redis://:${REDIS_PASSWORD}@redis:6379
```

```
      - NEXTAUTH_URL=${NEXTAUTH_URL}
      - NEXTAUTH_SECRET=${NEXTAUTH_SECRET}
      - RAZORPAY_KEY_ID=${RAZORPAY_KEY_ID}
      - RAZORPAY_KEY_SECRET=${RAZORPAY_KEY_SECRET}
      - RAZORPAY_WEBHOOK_SECRET=${RAZORPAY_WEBHOOK_SECRET}
      - MSG91_API_KEY=${MSG91_API_KEY}
      - MSG91_SENDER_ID=${MSG91_SENDER_ID}
      - GUPSHUP_API_KEY=${GUPSHUP_API_KEY}
      - SENDGRID_API_KEY=${SENDGRID_API_KEY}
      - FIREBASE_SERVICE_ACCOUNT=${FIREBASE_SERVICE_ACCOUNT}
      - UPLOAD_DIR=/app/uploads
      - NODE_ENV=production
    volumes:
      - uploads_data:/app/uploads
    ports:
      - "127.0.0.1:3000:3000"   # only accessible via Nginx
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy

volumes:
  postgres_data:
  redis_data:
  uploads_data:
```

6. Nginx Configuration

```
# nginx/conf.d/waterpark.conf

server {
    listen 80;
    server_name yourdomain.com www.yourdomain.com;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name yourdomain.com www.yourdomain.com;

    ssl_certificate     /etc/letsencrypt/live/yourdomain.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/yourdomain.com/privkey.pem;
    ssl_protocols       TLSv1.2 TLSv1.3;
    ssl_ciphers         ECDHE-RSA-AES256-GCM-SHA512:DHE-RSA-AES256-GCM-SHA512;

    # Security headers
    add_header X-Frame-Options "SAMEORIGIN";
    add_header X-Content-Type-Options "nosniff";
    add_header X-XSS-Protection "1; mode=block";
    add_header Referrer-Policy "strict-origin-when-cross-origin";
    add_header Permissions-Policy "camera=(), microphone=(), geolocation=()";

    # File upload size (for photo uploads)
```

```
    client_max_body_size 20M;

    # Proxy to Next.js
    location / {
        proxy_pass        http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header   Upgrade $http_upgrade;
        proxy_set_header   Connection "upgrade";
        proxy_set_header   Host $host;
        proxy_set_header   X-Real-IP $remote_addr;
        proxy_set_header   X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header   X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;
        proxy_read_timeout 300s;
    }

    # WebSocket support (for real-time features)
    location /api/v1/realtime {
        proxy_pass        http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header   Upgrade $http_upgrade;
        proxy_set_header   Connection "Upgrade";
        proxy_set_header   Host $host;
    }

    # Static file cache
    location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg|woff|woff2)$ {
        proxy_pass http://127.0.0.1:3000;
        expires 30d;
        add_header Cache-Control "public, immutable";
    }
}
```

7. Environment Variables (.env.example)

```
# Database
POSTGRES_DB=waterpark_db
POSTGRES_USER=waterpark_user
POSTGRES_PASSWORD=CHANGE_ME_STRONG_PASSWORD
DATABASE_URL=postgresql://waterpark_user:CHANGE_ME@postgres:5432/waterpark_db

# Redis
REDIS_PASSWORD=CHANGE_ME_REDIS_PASSWORD
REDIS_URL=redis://:CHANGE_ME@redis:6379

# App
NODE_ENV=production
NEXTAUTH_URL=https://yourdomain.com
NEXTAUTH_SECRET=CHANGE_ME_64_CHAR_RANDOM_STRING

# Razorpay
RAZORPAY_KEY_ID=rzp_live_xxxxxxxxxxxx
RAZORPAY_KEY_SECRET=xxxxxxxxxxxxxxxxxxxx
RAZORPAY_WEBHOOK_SECRET=xxxxxxxxxxxx
```

```
# MSG91 (SMS + WhatsApp)
MSG91_API_KEY=xxxxxxxxxxxx
MSG91_SENDER_ID=WTRPRK
MSG91_WHATSAPP_NUMBER=91XXXXXXXXXX

# Gupshup (WhatsApp - if using instead of MSG91)
GUPSHUP_API_KEY=xxxxxxxxxxxx
GUPSHUP_APP_NAME=waterpark
GUPSHUP_SOURCE_NUMBER=91XXXXXXXXXX

# Email
SENDGRID_API_KEY=SG.xxxxxxxxxxxx
SENDGRID_FROM_EMAIL=noreply@yourdomain.com
SENDGRID_FROM_NAME=Waterpark Name

# Firebase (Push Notifications)
FIREBASE_SERVICE_ACCOUNT={"type":"service_account",...}

# Encryption key for gateway secrets
ENCRYPTION_KEY=CHANGE_ME_32_CHAR_KEY

# File uploads
UPLOAD_DIR=/app/uploads
MAX_FILE_SIZE_MB=10
```

8. Application Dockerfile

```
# app/Dockerfile
FROM node:20-alpine AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --only=production

FROM node:20-alpine AS builder
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN npx prisma generate
RUN npm run build

FROM node:20-alpine AS runner
WORKDIR /app
ENV NODE_ENV production

RUN addgroup --system --gid 1001 nodejs
RUN adduser --system --uid 1001 nextjs

COPY --from=builder /app/public ./public
COPY --from=builder /app/.next/standalone ./
COPY --from=builder /app/.next/static ./.next/static
COPY --from=builder /app/prisma ./prisma
COPY --from=builder /app/node_modules/.prisma ./node_modules/.prisma
```

```
RUN mkdir -p /app/uploads && chown nextjs:nodejs /app/uploads

USER nextjs
EXPOSE 3000
ENV PORT 3000
ENV HOSTNAME "0.0.0.0"

CMD ["node", "server.js"]
```

9. Server Setup Commands (Ubuntu 22.04)

```
# ---- Step 1: Initial server setup ----
sudo apt update && sudo apt upgrade -y
sudo apt install -y curl git ufw

# ---- Step 2: Firewall ----
sudo ufw allow OpenSSH
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable

# ---- Step 3: Install Docker ----
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER
newgrp docker

# ---- Step 4: Install Docker Compose ----
sudo apt install -y docker-compose-plugin

# ---- Step 5: Install Nginx ----
sudo apt install -y nginx
sudo systemctl enable nginx

# ---- Step 6: Install Certbot ----
sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d yourdomain.com -d www.yourdomain.com

# ---- Step 7: Clone repo and setup ----
cd /opt
sudo git clone https://github.com/yourorg/waterpark-app.git
cd waterpark-app
cp .env.example .env
nano .env   # Fill in all values

# ---- Step 8: Start all containers ----
docker compose up -d

# ---- Step 9: Run database migrations ----
docker compose exec app npx prisma migrate deploy

# ---- Step 10: Seed admin user ----
docker compose exec app npx prisma db seed

# ---- Step 11: Copy Nginx config ----
```

```
sudo cp nginx/conf.d/waterpark.conf /etc/nginx/conf.d/
sudo nginx -t && sudo systemctl reload nginx

# ---- Verify all running ----
docker compose ps
docker compose logs app --tail=50
```

10. GitHub Actions CI/CD Pipeline

```
# .github/workflows/deploy.yml
name: Deploy to Production

on:
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Deploy via SSH
        uses: appleboy/ssh-action@master
        with:
          host: ${{ secrets.SERVER_HOST }}
          username: ${{ secrets.SERVER_USER }}
          key: ${{ secrets.SSH_PRIVATE_KEY }}
          script: |
            cd /opt/waterpark-app
            git pull origin main
            docker compose build app
            docker compose up -d app
            docker compose exec -T app npx prisma migrate deploy
            docker compose exec -T app npx prisma generate
            echo "Deployment complete: $(date)"
```

11. PostgreSQL Backup Script

```
#!/bin/bash
# scripts/backup-db.sh
# Run via cron: 0 2 * * * /opt/waterpark-app/scripts/backup-db.sh

DATE=$(date +%Y%m%d_%H%M%S)
BACKUP_DIR="/opt/waterpark-app/postgres/backups"
mkdir -p $BACKUP_DIR

docker compose -f /opt/waterpark-app/docker-compose.yml exec -T postgres   pg_dump -U $
```

```
# Keep only last 30 days
find $BACKUP_DIR -name "*.sql.gz" -mtime +30 -delete

echo "Backup completed: waterpark_$DATE.sql.gz"
```

12. Phase 1 Development Sprint Plan

Sprint	Duration	Goals
Sprint 1	Week 1-2	Project setup, Docker, DB schema, Prisma migrations, Auth (login/OTP/RBAC)
Sprint 2	Week 3-4	Admin panel shell, Employee panel shell, Role middleware
Sprint 3	Week 5-6	Ticket types config, Online booking flow (guest), Booking confirmation
Sprint 4	Week 7-8	Razorpay integration, Manual UPI + UTR submission, UPI verification queue
Sprint 5	Week 9-10	Walk-in POS (counter), Entry QR validation, Wristband issuance
Sprint 6	Week 11-12	F&B POS, Locker management, Admin dashboard (revenue, bookings)
Sprint 7	Week 13-14	Guest profile, CRM basics, MSG91 SMS/WhatsApp notifications
Sprint 8	Week 15-16	Reports module, Testing, Bug fixes, Production deployment